

红黑树与 B+ 树

2.1 红黑树

2.1.1 红黑树的引入与定义

为了实现平衡搜索树，AVL 树用一种非常简单暴力的思想，即保持严格平衡（任一结点左右子树高度差绝对值最多为 1）然后用旋转维持平衡。红黑树在某种程度上是希望放松这一假设，从另一角度——给结点染色，定义其它平衡因子——也给出了一种解决方案。首先来看红黑树的两条性质，或者说满足这两条性质的搜索树称为红黑树：

定义 2.1. 一棵红黑树是满足下面性质的二叉搜索树：

1. 每个结点或者是红色的，或者是黑色的；
2. 根结点是黑色的；
3. 每个叶结点（NIL）是黑色的；
4. 如果一个结点是红色的，那么它的两个子结点都是黑色的；
5. 对每个结点，从该结点到其所有后代叶结点的简单路径上，均包含相同数目的黑色结点。

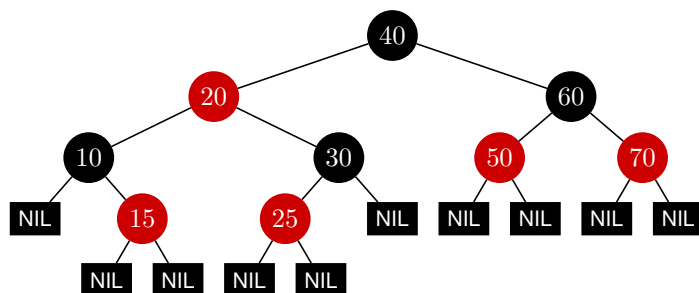


图 2.1 一棵红黑树示例，每个空孩子位置都画出了黑色的 NIL。为了图示简洁，有时我们不画出 NIL 结点。

在下面的讨论中，我们会用序号指代这五条性质。为方便讨论，假设树中各结点的键值互不相同。下面我们简要说明这五条性质的含义：（1）第一条的含义是红黑树只有这两种颜色，这一点或许很显然，但在之后讨论红黑树删除的时候有用处；（2）对于第二条，之后会经常遇到在插入、删除调整时，根结点变为红色的情况，此时如果直接将根结点染成黑色，对于其它四个条件是完全没有影响的；因此后面其实并不会太关心根结点的颜色问题，因为不是黑色直接染成黑色完全不影响其它性质；（3）第三条表明红黑树的叶结点是一类特殊的结点 NIL，它们不保存键值，且颜色为黑色；这或许有些迷惑，但既然定义如此必定有其作用，其作用一方面在于使得红黑树有一定的平衡性：如果没有 NIL 结点，完全 skew 的树也是可以符合其余几条规定的（如图 2.2 所示），另一方面作用将

会在讨论删除的时候体现。

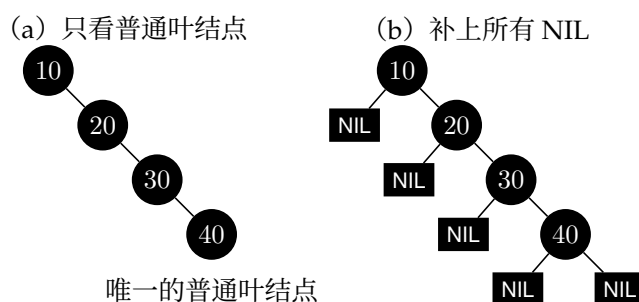


图 2.2 NIL 的作用。左图是一棵全部染黑的没有 NIL 结点的二叉搜索树，考察性质 5，由于每个结点下方都只有一条通往后代叶结点的路径，自然满足要求，且要求 1, 2, 4 也都满足，并且这样的链可以任意延长。右图补上 NIL 后，从根直接到左侧 NIL 的路径含两个黑结点，经过 20, 30, 40 到 NIL 的路径含五个黑结点（均含根和末端 NIL），违反性质 5。

对于最后两条定义，它们的意义在于保证红黑树的树高是 $\log n$ 级别的，从而红黑树是平衡搜索树。在介绍定理之前，需要先引入两个定义：

定义 2.2. 1. 称 NIL 为外部结点，其余有键值的结点为内部结点。

2. 从某个结点 X 出发到达一个叶结点（NIL）的任意一条简单路径上的黑色结点个数（不含 X 本身，包含 NIL）称为 X 的黑高，记为 $bh(X)$ 。根据定义第五条，这一定义是合理的，因为从 X 出发到达一个叶结点的任意一条简单路径上的黑色结点个数相同。除此之外，定义整棵红黑树的黑高为其根结点的黑高。

需要注意的是，NIL 结点的黑高为 0。在这一定义的基础上，下面给出红黑树的高度上界，从而说明红黑树是平衡搜索树。

定理 2.3. 一棵有 n 个内部结点的红黑树的高度至多为 $2\log(n+1)$ 。

证明. 先证明以任意结点 X 为根的子树至少有 $2^{bh(X)} - 1$ 个内部结点。对树的结构归纳： $X = \text{NIL}$ 时内部结点个数为 0，符合结论。设 X 是内部结点，黑高为 b ；其每个孩子的黑高为 b （孩子红）或 $b-1$ （孩子黑）。由归纳假设，加上 X 自己，内部结点数至少为

$$1 + 2(2^{b-1} - 1) = 2^b - 1.$$

故 $n \geq 2^{bh(\text{root})} - 1$ ，即 $bh(\text{root}) \leq \log_2(n+1)$ 。

再看一条从根到 NIL 的路径，不计根而计末端 NIL。由于红结点不能相邻，且末端是黑色，红结点数不会超过黑结点数，所以路径边数至多为 $2bh(\text{root})$ 。此外，空树单独满足结论。□

总结而言，对于红黑树，黑高是绝对严格的平衡要求，而红色结点则是少量不平衡的因素，定义控制了红色结点的个数，也就控制了不平衡因素的影响，因此红黑树还是可以保持一定程度的平衡的。有了这一结论，至少现在可以知道对于搜索操作，红黑树的时间复杂度是 $O(\log n)$ 的，至于插入和删除，还需要下面进一步讨论。

2.1.2 红黑树的插入

与 AVL 树类似，插入结点时有可能出现平衡性质被破坏，因此需要一定的调整。那么自然面临的第一个问题就是：插入应该插入黑色结点还是红色结点？假设每次都插入黑色结点，那么定义第五条一定会被破坏，因此每次插入都必须调整（除了从空树插入的那次）；但是如果插入红色结点，定义第二条（仅在插入空树时）和定义第四条仅仅是有可能被破坏，因此直观来看插入红色结点会比插入黑色结点需要的调整更少。除此之外，在学完红黑树删除后，红黑树删除黑色结点时的复杂情况也暗示了插入红色应当是更好的选择。

接下来开始介绍红黑树的插入操作，根据前面的讨论，插入的结点首先都是红色的。最简单的情况是插入在黑色结点下面，这时无需任何调整，因为没有破坏任何性质；其次是插入空树时，这时直接把结点染黑即可恢复定义第二条。因此下面只讨论最复杂的情况，即插入到了红色结点的子结点的位置。此时唯一被违反的就是定义第四条。因此如果要解决这一问题，自然的想法就是，在不影响其它平衡性质（主要是第五条）的前提下，通过一些染色和旋转使得没有父子都是红色。主要分为以下三种情况讨论（假设插入的结点为 X ，先讨论 X 插入在祖父 G 左侧的情况）：

情况 1：X 的叔叔（即父亲的兄弟）是红色的，X 无论左右孩子都是该情况

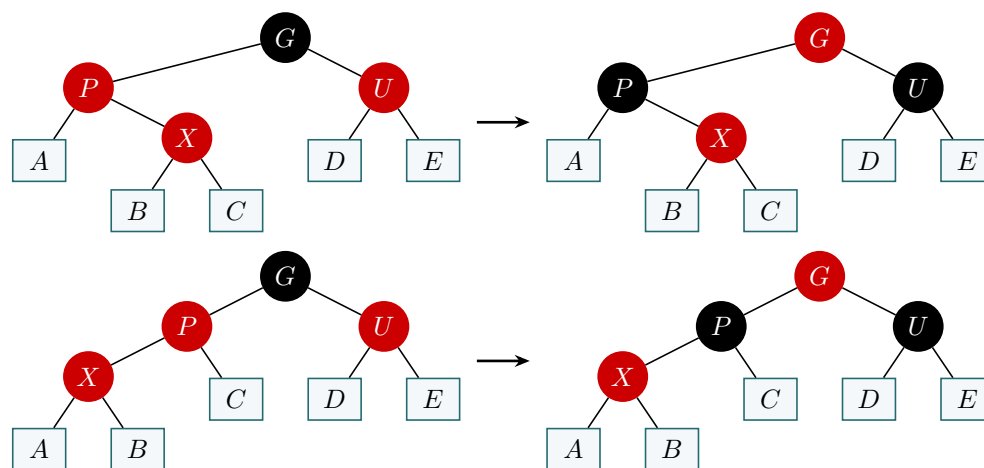


图 2.3 叔叔为红：只染色，将问题交给祖父。

需要注意的是，这里所有结点都带子树（图中浅底方框），一方面至少有 NIL 结点，另一方面这可以不仅仅表示刚刚插入的情况，也可以表示经过几次调整后还在被这种情况困扰，因此更具一般性。注意 G 一定是黑色，因为 P 是红色，插入前它们就在红黑树中，因此不可能违背定义第四条。

下面开始解释解决方案，事实上此时解决方案非常直白：它的父亲和叔叔都是红色，如果把红色看成一种 debuff， X 就不能把这个 debuff 甩给他们，只能求助于祖父，于是直接把 X 的父亲和叔叔染黑，祖父染红。此时完全不影响黑高性质，但问题并没有解决，因为祖父的父亲可能还是红色！但是，问题被往上推了，最差的情况也是一直上推到把问题交给根结点，那么根结点此时只需要直接染黑就可以解决问题。当然也可能问题上推后变为下面两种情况，接下来我们进行解释：

情况 2：X 的叔叔（即父亲的兄弟）是黑色的，且 X 是右孩子

情况 3：X 的叔叔（即父亲的兄弟）是黑色的，且 X 是左孩子

情况 2 中， P 是 G 的左孩子， X 是 P 的右孩子，与 AVL 树的 LR 情形相同。先对 P 左旋，使 X 接替 P 的位置，这一步保持颜色不变。旋转后，两个相邻的红结点都位于左孩子方向，便转成了

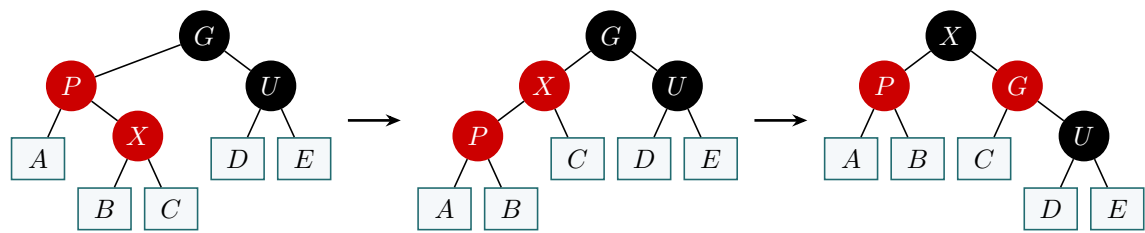


图 2.4 叔叔为黑：先从情况 2 转到情况 3，再旋转并染色完成修复。

情况 3。接着对 G 右旋，将接替 G 的结点 X 染黑，再将 G 染红。整个过程中，不难验证搜索树性质和黑高都得以保持。

当然还有 X 插入在祖父 G 右侧的情况，这是完全对称的，因此不再重复。总结而言，就是第一种情况叔叔红，直接改颜色，问题推给祖父，第二、三种情况叔叔黑且两个连续的红色构成类似 AVL 树的 LR、LL（对称的有 RL、RR）结构，就用 AVL 的旋转，最后染色得到根黑色，第二层红色，大致流程如下：

当前情况	调整后的去向
1	继续检查祖父：可能仍为 1，或进入 2、3，或结束
2	一次旋转，进入 3
3	一次旋转并染色，结束

根据这一流程可知，红黑树插入最多可能的旋转次数为 2，因为只有情况 2 和 3 会要旋转，且进入情况 2 后 1 次旋转必定进入情况 3，进入情况 3 后 1 次旋转必定解决；更改颜色最多是 $O(\log n)$ 次，因为进入情况 2 或 3 只需要一次染色，在情况 1 最差也是每两层染一次色，而已经证明红黑树的最大高度是 $O(\log n)$ 的。因此插入操作包括 $O(\log n)$ 的搜索时间，加常数的旋转，加 $O(\log n)$ 的染色，因此还是 $O(\log n)$ 的时间复杂度，总结而言有如下结论：

定理 2.4. 一棵有 n 个内部结点的红黑树插入一个结点的时间复杂度为 $O(\log n)$ 。

图 2.5 给出了一棵红黑树插入 65 后的修复过程，先在下层染色，再在上层依次右旋、左旋并染色，连续经过三种修复情况。

读者可以根据上述流程，尝试自行用代码实现红黑树的插入操作，此处限于篇幅不展开讨论。

2.1.3 红黑树的删除

红黑树的删除是一个并不简单的操作，这其中有太多可能的情况需要不同的调整策略，接下来尽可能地清晰列举。在讲解红黑树删除之前，再次回顾普通平衡搜索树的删除操作，假设被删除的结点为 X ：

1. 如果 X 没有孩子，直接删除即可；
2. 如果 X 只有一个孩子，让孩子接替 X 的位置；
3. 如果 X 有两个孩子，那就让 X 与其左子树的最大结点（或右子树最小结点）交换，然后删除 X

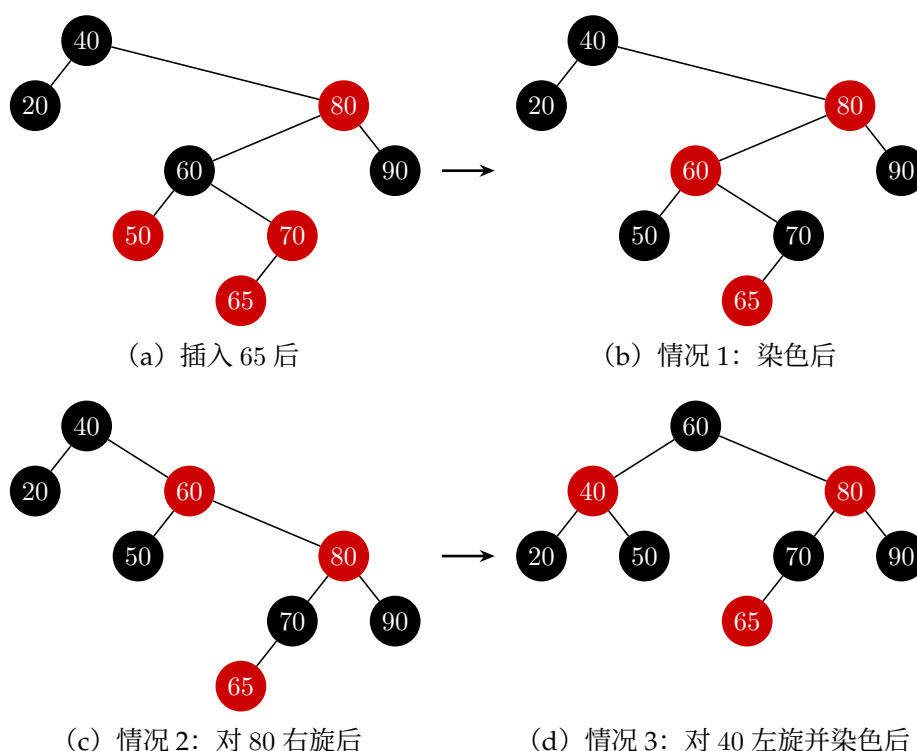


图 2.5 插入 65 后，先在下层染色，再在上层依次右旋、左旋并染色，连续经过三种修复情况。

(这时 X 所在的位置一定最多只有一个子结点，因为左子树最大结点不可能有右孩子，右子树最小结点不可能有左孩子)。

红黑树的删除是基于这些操作的，需要注意的是第三种情况， X 和与其交换的结点只交换键值及其附属记录，不交换颜色，否则如果两者颜色不同，在交换的时候就可能破坏第五条性质，这是很难令人满意的。由于第三种情况可以通过一步交换直接转化为第一或第二种情况，因此只需要关心第一和第二种情况。在第一种情况中，接替被删除结点所在位置的结点是 NIL，第二种则是被删除结点的子结点。如果被删除的结点是红色，事实上无事发生，没有任何性质被破坏；如果被删除的是黑色，如果接替上来的结点是红色的，直接染黑也不会破坏任何性质。接下来就是问题的关键，如果接替的是 NIL 或是黑色结点应该怎么办？

这个办法很聪明：直接给黑色结点或者 NIL（其实也是黑色结点）再加一重黑色，于是它的颜色变成了“双黑”。此时第五条性质没有被破坏，但是，第一条性质被破坏了：这里出现了非红也非黑的颜色！于是想法就是：把这个黑色的 debuff 扔给一个红色结点，或者一步一步往上扔给根结点，因为根结点把双黑直接变成黑色完全不影响其它性质。

下面假设 X 是父结点 P 的左孩子，它的兄弟为 S 。把 S 的左孩子记为 N_1 ，右孩子记为 N_2 ；由于 N_1 靠近 X ， N_2 远离 X ，分别称它们为近侄子和远侄子。下面的图中，浅蓝结点的原色可以是红或黑，外加一圈表示额外一重黑色，因此黑色双圈就是双黑。注意此时兄弟和侄子一定非空，因为 X 是双黑，所以从 S 开始还有至少 2 个黑结点（包括 S 和 NIL）。接下来的讨论中 X 都是父亲的左孩子，因为右孩子情况对称故不重复讨论：

情况 1: X 的兄弟是红色的

在这一情况下，由于原先的树满足红黑树定义第四条，因此 P 一定是黑色。我们的想法很简单，兄弟是红色，那就希望兄弟能两肋插刀，把兄弟转上去，为了保持黑高性质，只能把父亲染红，并把 S 染黑避免出现连续的红色结点，自己仍承受双黑 debuff。但是好处在于，这个问题转化为了接下

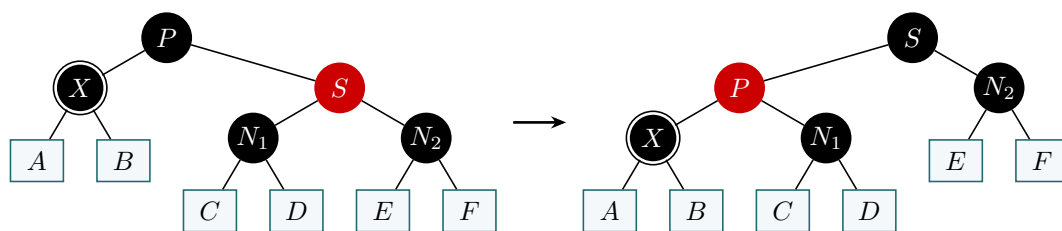


图 2.6 删除情况 1: 红兄弟转上去, 新兄弟为黑色。

来的情况二三四中的一种, 下面来看如何解决。

情况 2: X 的兄弟是黑色的, 且兄弟的两个孩子都是黑色的

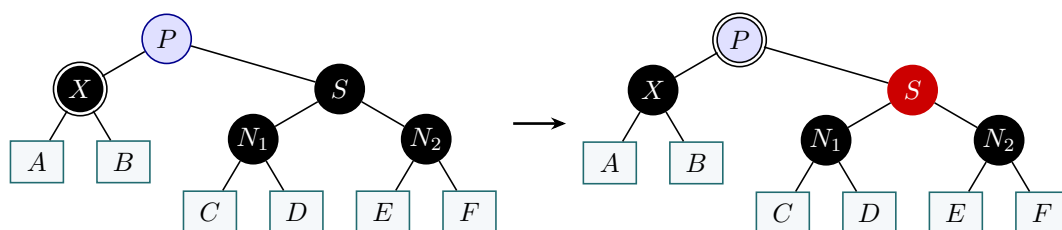


图 2.7 删除情况 2: 额外黑色交给父亲。

兄弟一侧没有红色侄子可用, 就把额外黑色往上推给父亲, 为了保证红黑树性质, 将兄弟从黑变红, 总而言之就是 X 这一层的黑色全部往上推。如果父亲原本是红色, 那就染黑, 问题解决; 如果父亲原本是黑色, 那父亲就变成双黑, 让问题向根结点靠近。需要特别注意的是, 如果是从情况 1 变成情况 2 的, 父亲一定是红色, 所以如果是 1 变为 2, 则问题会马上解决。

情况 3: X 的兄弟是黑色的, 且近侄子是红色的, 远侄子是黑色的

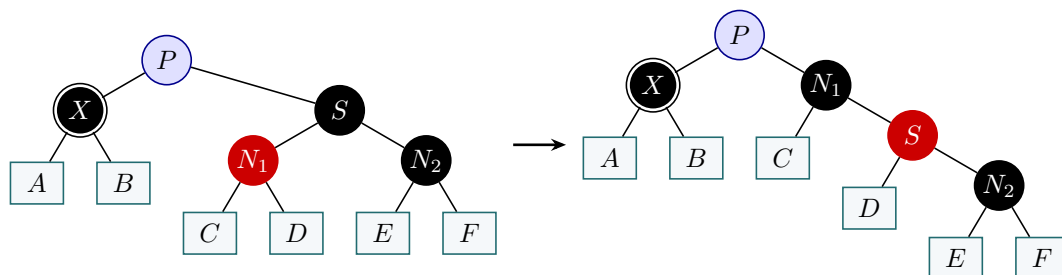


图 2.8 删除情况 3: 先把近侧红侄子转上去, 得到远侧红侄子。

这时借用 AVL 树的想法, 红色在父亲 P 的 RL 位置, 因此做 double rotation: single rotation 后会变成情况 4 的 RR 的情况。注意这里需要染色, 其目的是产生 RR 的情况。

情况 4: X 的兄弟是黑色的, 且远侄子是红色的, 近侄子颜色任意

此时对应 AVL 树的 RR, 于是再一次 single rotation 即可把双黑的一重黑丢给红色远侄子 (即 X 和 N_2 都变成黑色), 但要注意为了保证黑高性质, 即 P 和 S 还要交换颜色, 此时问题解决。

据此可以总结并计算出删除操作的时间复杂度。首先最多用 $O(\log n)$ 的时间找到删除结点, 最多 1 次交换和 1 个删除的操作。接下来如果删除后没有问题则到此结束; 否则根据流程, 情况 1、3 和 4 在问题解决前最多进去一次, 因为 4 可以直接解决, 3 直接进入 4 然后解决, 1 如果进入 3 和 4 也可以马上解决, 进入 2 后也因为父结点是红色可以马上解决。因此关键在于情况 2 可能出现很多

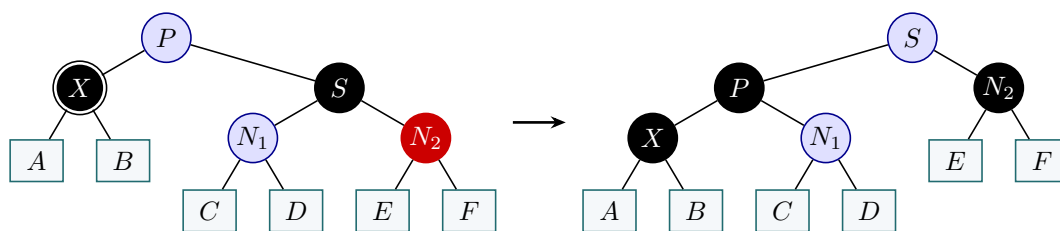


图 2.9 删除情况 4: 旋转并补足黑色数，额外黑色消失。

当前情况	调整后的去向
1	进入 2、3 或 4；若进入 2，则立即结束
2	父亲红则结束；父亲黑则向上继续，或到根结束
3	一次旋转，进入 4
4	一次旋转并染色，结束

次，但最多也只是树高 $O(\log n)$ 次，因为每次都会上推 1 格。总而言之，因为情况 1、3 和 4 在问题解决前最多进去一次，所以最多 3 次旋转加上 $O(\log n)$ 次颜色调整可以解决问题，因此有如下结论：

定理 2.5. 一棵有 n 个内部结点的红黑树删除一个结点的时间复杂度为 $O(\log n)$ 。

删除算法的具体代码实现读者可以自行尝试，需要提醒的是双黑这一性质无需额外的字段表示，因为可以始终保证当前考虑的结点 X 就是双黑的结点，所有条件判断等都围绕 X 进行。

接下来我们看一个具体的例子。在图 2.10 的初始树中，先删除 10，再删除 70，这两次删除会用到前面的全部四种情况。图中只画出带额外黑色的 NIL，其余 NIL 省略。

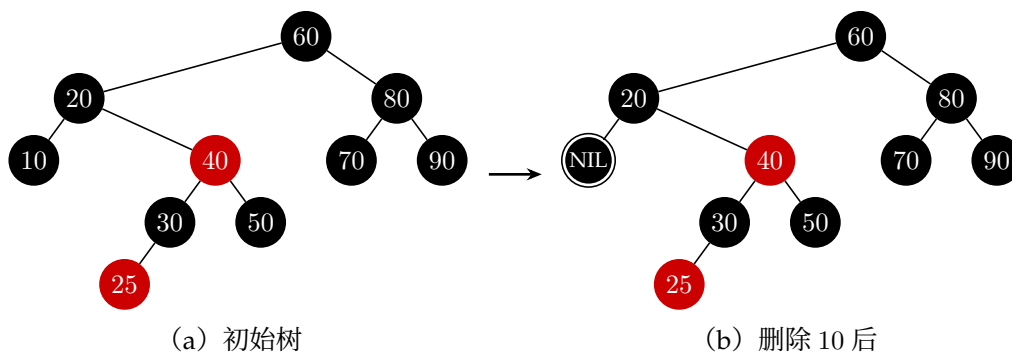
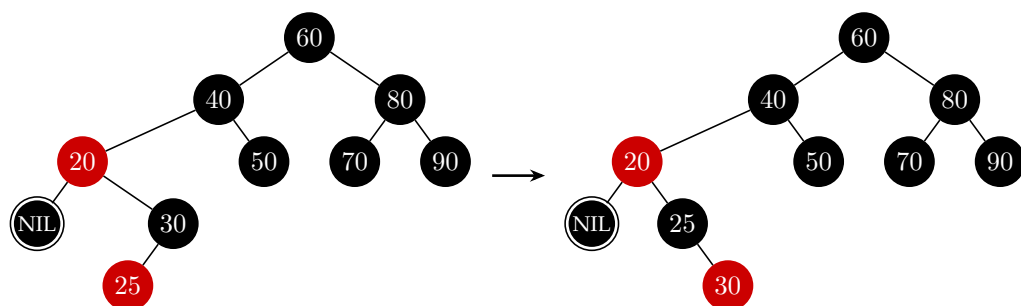


图 2.10 第一次删除：从初始树删去黑色叶子 10，其位置留下双黑 NIL。

删除 10。 结点 10 是黑色叶子，删去后，接替它的 NIL 成为双黑结点 X 。此时进入情况 1。对 20 左旋，并把 40 染黑、20 染红，就得到图 2.11 (c)，进入情况 3。因此对 30 右旋，并把 25 染黑、30 染红，得到 (d)。现在进入情况 4。对 20 左旋，让 25 继承 20 的红色，再把 20 和 30 染黑，同时去掉 X 的额外黑色，就得到图 2.12 (e)。至此，删除 10 连续经历了情况 1、3、4，共做了三次旋转。

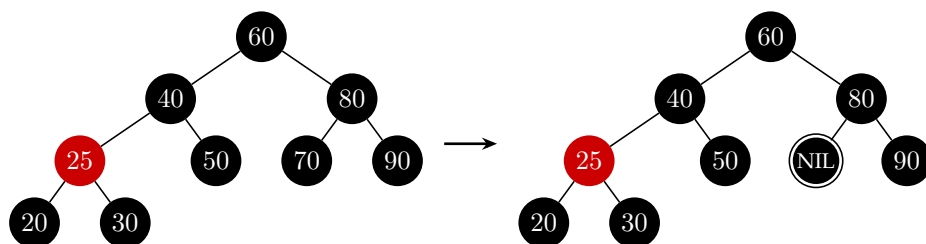
删除 70。 这次留下的双黑 NIL 是 80 的左孩子。它的兄弟 90 是黑色，两个侄子都是黑色 NIL，所以进入情况 2，如 (f) 所示。把 90 染红，父亲 80 成为新的双黑结点 X ，还需要继续调整。图 2.13 (g) 符合情况 4，于是对 60 右旋，把远侄子 25 染黑，再去掉 80 的额外黑色，就得到 (h)。



(c) 情况 1: 对 20 左旋并染色

(d) 情况 3: 对 30 右旋并染色

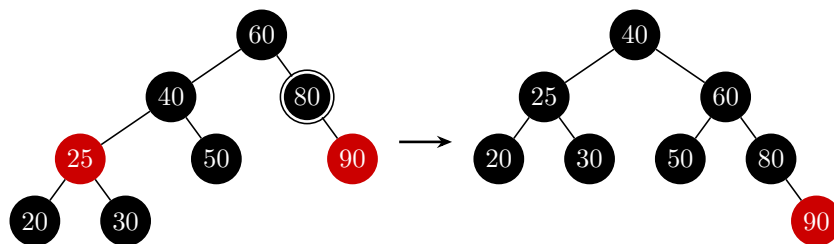
图 2.11 情况 1 之后, 新兄弟满足情况 3; 再次旋转后, 红色结点 30 成为远侄子。



(e) 情况 4: 对 20 左旋并染色

(f) 接着删除 70

图 2.12 (e) 删除 10 并恢复平衡后的树; (f) 在此基础上删除 70, 进入情况 2。



(g) 情况 2: 额外黑色移到 80

(h) 情况 4 (对称): 对 60 右旋并染色

图 2.13 情况 2 把额外黑色交给 80; 随后使用情况 4 的操作, 完成第二次删除。

AVL 树和红黑树的比较

本节我们比较 AVL 树和红黑树这两类平衡搜索树。首先两种树都通过旋转恢复平衡, 但约束不同。AVL 树的高度上界约为 $1.44 \log_2 n + O(1)$, 红黑树为 $2 \log_2(n+1)$, 因此 AVL 的最坏搜索路径上界更小。另一方面, AVL 删除可能沿祖先链做 $O(\log n)$ 次旋转, 本讲红黑树的插入、删除则分别至多旋转两次、三次 (但染色仍可能有 $O(\log n)$ 次)。这些差别会影响实际性能, 需要根据应用场景选择, 例如查找时 AVL 树因为平衡性更好可能更快, 而更新时因为红黑树删除旋转次数更少可能更快。

然而, 以上讨论只是大致的直观, 实际执行时可能红黑树的平衡性也不错, AVL 树的旋转次数也不一定特别多。当然一个有趣的事实是, GCC 的 libstdc++ 使用红黑树作为几类 STL 容器的基础, Microsoft 标准库也采用红黑树实现。此外, 内存布局也是一个重要的影响因素: AVL 的平衡因子只有三个稳定取值, 理论上需要 2 bits 存储; 红黑树颜色只需 1 bit, 但父、左、右指针、键和值, 以及对象对齐所需的空间, 都会影响内存大小。因此实际性能并没有严格的哪个更好的结论, 更多是取决于应用场景和实现细节。

2.2 B+ 树

2.2.1 B+ 树的定义

接下来要介绍的 B+ 树使用了另一种思路来改进操作的时间复杂度。二叉搜索树每个结点最多有两个孩子，B+ 树则允许每个结点有多个孩子。这样可以减少树的高度，进而在数据量非常巨大的时候（例如现实中的数据库系统）大大减少操作所需的磁盘访问次数。B+ 树还有一个特点是所有的记录（数据）都存储在叶结点中，内部结点（非叶结点，包括非叶的根结点）只存储索引信息，这样可以使得范围查询更加高效。

在给出定义之前，先说明 B+ 树中存储的记录是什么。例如，一条学生数据包含“学号：20240001，姓名：张三，专业：计算机”，这一整条数据称为一条记录。如果按学号建立索引，那么学号 20240001 就是这条记录的键，姓名和专业则是记录中的其他字段。在本讲中，我们要求键互异，记录按键排序。B+ 树根据键的比较结果选择查找路径，因此，为了看清树的结构和操作过程，图中通常只画记录的键，省略其他字段。

定义 2.6. 满足如下性质的树称为阶数 $M \geq 3$ 的 B+ 树：

1. 根要么是叶子，要么有 2 到 M 个孩子；非根内部结点有 $\lceil M/2 \rceil$ 到 M 个孩子。
2. 非根叶子有 $\lceil M/2 \rceil$ 到 M 条记录；根若是叶子，可以有 0 到 M 条。
3. 所有叶子深度相同，叶子按键递增连接成链。
4. 若内部结点的孩子从左到右为 $c_1, \dots, c_q$ ，则存 $q - 1$ 个索引。其中第 i 个索引等于以 c_{i+1} 为根的子树中所有记录的最小键值（ $1 \leq i < q$ ），用于查找时选择孩子。

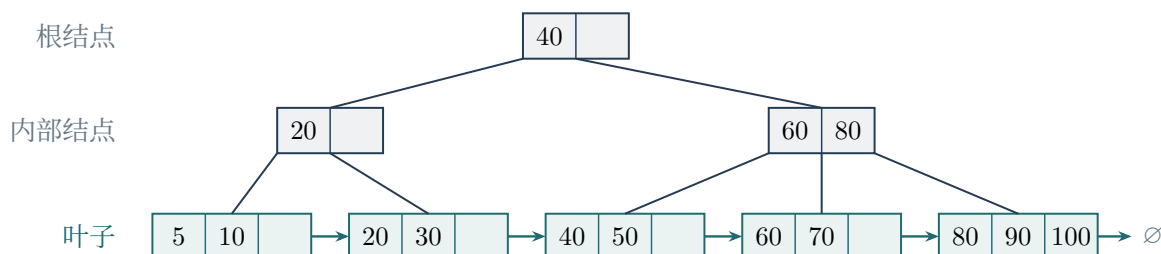


图 2.14 $M = 3$ 的三层 B+ 树示例：内部结点有两个或三个孩子，叶子有两条或三条记录。

有时我们也称阶数 $M \geq 3$ 的 B+ 树为 $\lceil M/2 \rceil - \dots - M$ 树，例如 $M = 3$ 的 B+ 树也称为 2-3 树， $M = 4$ 的 B+ 树也称为 2-3-4 树。图 2.14 给出了一个 B+ 树的例子。图中用方格表示结点中的位置，空格表示尚未使用的位置。内部结点保存索引和孩子指针，叶子则保存记录。叶子间的水平箭头表示指向后继结点的指针，最后一个叶子的后继为空。因此所有叶子实际上构成了一个链表，这一链表的作用在介绍范围查询时会理解。

关于这一定义还有几点需要进一步解释

1. 非根内部结点的孩子数和非根叶子的记录数都要求在 $\lceil M/2 \rceil$ 到 M 之间，因此当 M 取较大值时，B+ 树的高度会很小，从而保证查找路径的长度较短，否则当数据量巨大时，在磁盘中查找一条记录可能需要多次磁盘访问，效率很低。实际上，现实数据库的 M 可能达到数百甚至上千。
2. 对于根结点的要求并不严格，因为如果根结点就是叶结点时，说明整棵树只有这一个结点，因此

数据量可以从 0 开始；此外，当数据量逐渐增大时，根据后续介绍的插入操作，根结点可能会首先分裂产生两个孩子，因此根结点的孩子数下限为 2。

3. B+ 树的记录只存在叶子中，内部结点只保存键的索引，并且第 i 个键是从左到右第 $i + 1$ 个孩子的最小键。因此 M 阶 B+ 树非根内部结点中的索引个数范围是 $[M/2] - 1, \dots, M - 1$ 。例如图 2.14 中，根的索引 40 是其第二棵子树（右子树）的最小键，其余索引同理。

2.2.2 查找与范围查询

在理解了 B+ 树的定义后，接下来介绍其基本操作。注意本节的算法思路都比较基本，因此不提供伪代码。但真实实现 B+ 树的时候还是有很多复杂的细节，感兴趣的读者可以自行尝试。

首先是查找操作。查找操作从根结点开始，比较索引选择孩子，直到到达叶结点。根据 B+ 树的定义，如果查找的值小于第一个分割键，则选择第一个孩子；如果查找的值大于等于最后一个分割键，则选择最后一个孩子；如果查找的值大于等于第 i 个分割键且小于第 $i + 1$ 个分割键，则选择第 $i + 1$ 个孩子 ($i = 1, \dots, q - 2$)。到达叶结点后，再在叶结点中查找记录。例如图 2.14 中查找 30，先在根比较，因为 $30 < 40$ ，走左孩子；再与索引 20 比较，走右孩子，然后在叶子中找到记录。

当然我们还可以进行范围查询。若查询区间 $[15, 45]$ ，先根据 15 查找相应的叶子，从其中第一个不小于 15 的键开始，沿叶子链表依次读出 20, 30, 40，遇到 $50 > 45$ 后停止。由此我们看到叶子形成链表的作用。

需要注意的是，在具体实现时，因为 M 的值可能很大，所以在所有结点中寻找键值时都应该使用二分查找，因为结点中的键值是有序的。

2.2.3 插入

插入与搜索一致，首先按搜索这一目标键的方式找到相应的叶结点，然后把记录插入到叶结点中。如果叶结点的记录数没有超过上限 M ，则直接插入即可。此时也无需更新父结点的索引，因为根据搜索原则，要么叶结点的最小键没有变化，要么插入的是整棵树的最小键，此时父结点的索引也不变。

如果叶结点的记录数超过了上限，则需要进行分裂操作。分裂操作是将当前结点的记录按键排序后，从中间切成两份，左半部分保留在原结点，右半部分放入新创建的结点中，并将新结点插入到父结点中。如果父结点也超过了上限，则继续向上分裂，直到根结点为止。如果根结点也超过了上限，则创建一个新的根结点，将原来的根和新创建的结点作为其孩子。如果 M 是偶数，可能出现分裂后两份的记录数不相等的情况，此时没有严格的要求，左半和右半的记录数可以相差 1。分裂后，父结点的索引需要更新为新孩子的最小键。

考虑图 2.15 中向阶 $M = 3$ 的 B+ 树插入 35 的例子。首先叶子 $[20, 25, 30]$ 变成 $[20, 25, 30, 35]$ ，分成 $[20, 25]$ 和 $[30, 35]$ ；父亲因此有四个孩子，也要分裂。

2.2.4 删除

删除的第一步仍然是找到叶结点，然后删去对应的记录。如果删去后叶子记录数仍够，只更新索引即可。此处要更新索引是因为有可能删除的是某个父结点的子树的最小键。如果叶子记录数少于下限，则首先检查同一父亲下的左、右兄弟有没有超过下限的记录数。如果左兄弟有余量，就拿它的最后一个条目放到自己最前面；右兄弟有余量，就拿它的第一个条目放到自己最后，然后更新索引。

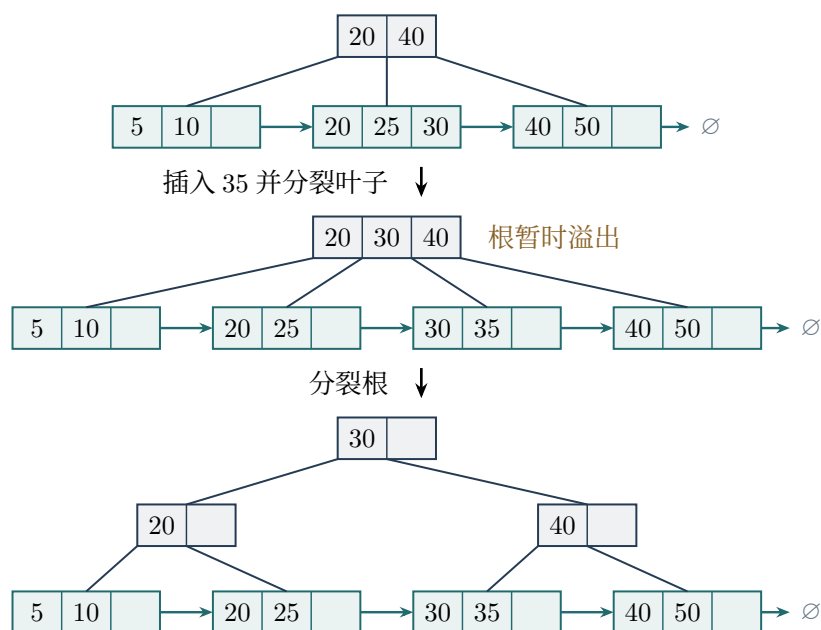


图 2.15 插入 35 引起叶子分裂和根分裂。箭头之间给出完整状态。

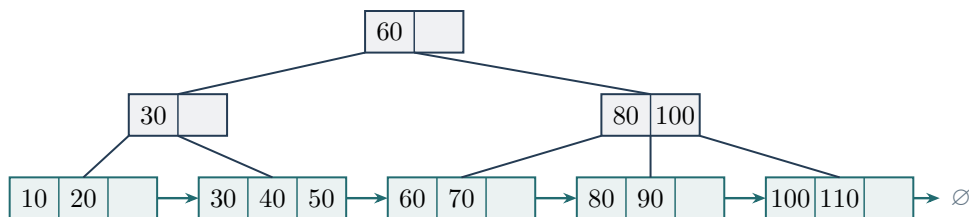
如果兄弟都刚好达到下限，只能合并兄弟结点。由于记录数不足的结点有 $\lceil M/2 \rceil - 1$ 个记录，与有 $\lceil M/2 \rceil$ 个记录的兄弟合并后共有 $2\lceil M/2 \rceil - 1 \leq M$ 条记录，可以放下（这里也体现了为什么 B+ 树定义中要用 $\lceil M/2 \rceil$ 作为下界）。注意此时父亲会因此少一个孩子，也许需要继续借或合并，注意此时的借就不再是借记录了，而是直接借一整个孩子。逐步向上处理，直到根结点为止。如果根只剩一个孩子，则孩子接替根即可。

图 2.16 在同一棵 $M = 3$ 的 B+ 树上依次删除 10, 20, 100。首先删除 10 后，最左叶子只剩 20，而右兄弟 $[30, 40, 50]$ 有三条记录，因此把记录 30 移到最左叶子并更新索引即可，如图 (b) 所示。接着删除 20，此时右兄弟已不能再借出记录。将两者合并成 $[30, 40, 50]$ 后，它们的父结点只剩一个孩子，也低于下限，但这个父结点的右兄弟有三个孩子，可以把最左边的整个叶子 $[60, 70]$ 交给它，然后修改索引即可，如图 (c) 所示。最后删除 100，最右叶子只剩 110，与左兄弟合并成 $[80, 90, 110]$ 。它们的父结点因此只剩一个孩子，而左边的内部结点也只有两个孩子，无法借出，于是合并这两个内部结点并更新索引。此时根只剩这一个孩子，就由孩子接替原根，如图 (d) 所示。

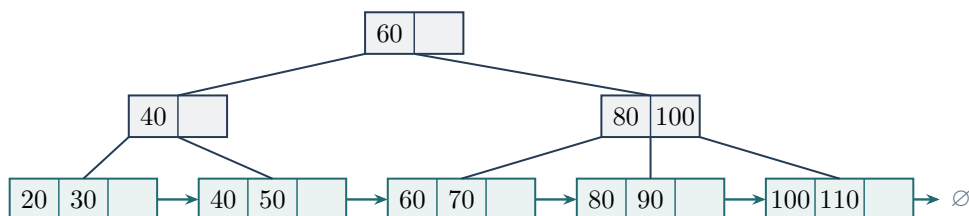
更新索引的具体算法

事实上，更新索引是一个听起来很简单，但在实现时需要特别注意的细节问题。一种最 naive 的想法是，在每次插入或删除结束后，从每个内部结点出发向下搜索相应的最小键，然后更新索引，但这样做显然不够高效。

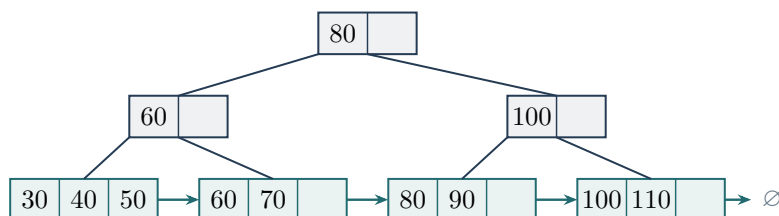
一种合理的实现方式是为每个结点储存一个 MinKey 字段，对于叶结点而言就是其中的最小键，对于内部结点而言就是其子树中的最小键。这样在插入或删除时，只需要在插入或删除的同时，在叶结点中更新 MinKey，然后沿着父亲链向上更新的过程中，每个父结点只需要根据 B+ 树的定义提取其相应子树的 MinKey 然后根据其更新索引即可。这样由于每一层在插入或删除时最多只有 2 个结点需要更新索引，每个结点有 $O(M)$ 个索引，因此更新索引的时间复杂度是 $O(M \log N)$ 。

(a) 初始状态: $M = 3$ 

(b) 删除 10: 向右兄弟借记录 30



(c) 删除 20: 合并叶子后, 父结点向右兄弟借一个孩子



(d) 删除 100: 合并叶子、合并内部结点, 原根撤去

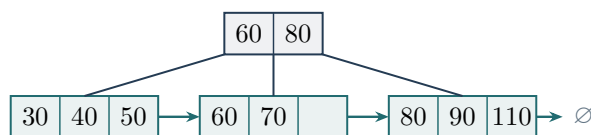


图 2.16 在同一棵 B+ 树上依次删除 10, 20, 100。各子图给出一次删除及其调整全部完成后的状态。

2.2.5 CPU 时间与磁盘访问

在理解了 B+ 树的基本操作后, 我们分析其时间复杂度。假设 B+ 树中有 N 条记录, 阶数为 M 。记 $b = \lceil M/2 \rceil$, 树高为 h 。当根不是叶子时, 根据 B+ 树的定义, 至少有 $2b^{h-1}$ 个叶子, 每个叶子至少有 b 条记录, 故 $N \geq 2b^h$, 进而有 $h \leq \log_b(N/2)$ 。综合考虑空树的情况, 从根结点走到任意一个叶子访问的总结点数符合 $O(1 + \log_M(N + 1))$ 。

首先考虑查找单个元素的复杂度。因为每层二分查找最多比较 $O(\log M)$ 次, 因此根据对数换底公式, 查找的时间复杂度为 $O(\log M(1 + \log_M(N + 1))) = O(\log(N + 1))$ 。区间查找的时间复杂度为 $O(\log(N + 1) + N)$, 因为要在叶结点中按顺序查找输出最多 N 条记录。

然后考虑插入、删除的复杂度。由于插入、删除时每层移动和更新最多 $O(M)$ 个条目, 因此时间为 $O(M(1 + \log_M(N + 1)))$ 。当然插入和删除首先要查找相应的叶结点, 但这部分的时间复杂度是不超过 $O(M(1 + \log_M(N + 1)))$ 的, 因此总的时间复杂度仍然是 $O(M(1 + \log_M(N + 1)))$ 。

基于上述讨论, 我们可以总结三类搜索树 (AVL 树、红黑树和 B+ 树) 的最坏时间如下表所示 (其中 n, N 分别是键或记录数)。

结构	查找	插入	删除
AVL	$O(\log(n+1))$	$O(\log(n+1))$	$O(\log(n+1))$
红黑树	$O(\log(n+1))$	$O(\log(n+1))$	$O(\log(n+1))$
B+ 树	$O(\log(N+1))$	$O(M(1+\log_M(N+1)))$	同插入

对于 B+ 树来说，其中存储的数量级可能非常大，因此通常在外存（磁盘）中存储。所以除了上面的时间复杂度之外，我们还需要考虑磁盘访问引入的 I/O 成本，此前讨论的时间复杂度则称之为 CPU 时间。由于外存一次读写以一个页（page）为单位，将一页读入内存以后的操作则相对快非常多。若一个结点恰好占一页，查找时就只需每层读一页，I/O 次数是 $O(1+\log_M(N+1))$ ，插入和删除也只在每层读写最多常数个结点。由此可以看出，把同一页放得更满、让树更矮，就能减少外存访问，从而提高性能。

2.2.6 B 树、B+ 树与 B* 树

事实上，B+ 树是 B 树族的一种。在 B 树族中还有 B 树和 B* 树。B 树与 B+ 树都是多路平衡搜索树，所有叶子的深度相同，主要区别在于记录存放的位置。B 树的内部结点和叶子都可以保存记录（或指向记录的指针），所以查找可能在内部结点结束。B+ 树则把所有记录集中在叶子中，内部结点只保存索引和指向孩子的指针，即使在内部结点遇到相等的键，也要继续走到叶子才能取得记录。图 2.17 用同一组记录展示了这一区别。

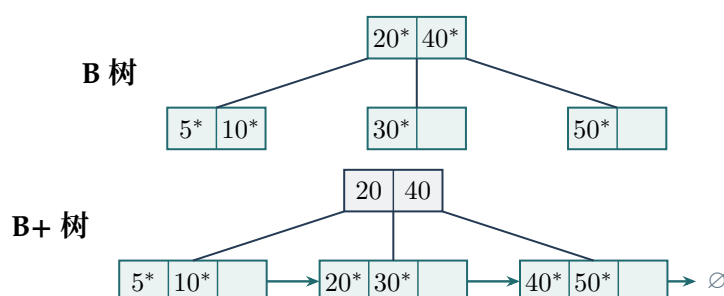


图 2.17 同一键集合的 B 树与 B+ 树。星号表示此处保存记录（或指向记录的指针）。

这样安排后，B+ 树的内部结点不必保存记录或记录指针，在相同的空间（一页）内通常可以容纳更多孩子，从而可能可以降低树高。此外，B+ 树的叶子按顺序连接，范围查询只需找到起点，再沿叶子链读取，B 树的记录分布在各层，范围查询需要按序遍历相应的子树。

B* 树主要改进结点的空间利用率：插入导致结点溢出时，先尝试与同一父亲下的相邻兄弟重新分配条目；若兄弟也已满，就把两个结点的条目连同新增条目分配到三个结点中。因此，分裂后的结点能保持约三分之二满，比通常的半满下限更高。当然删除的操作也要配套调整，例如将三个相邻结点合并为两个。

2.3 补充材料：红黑树与 2-3-4 B 树

学习红黑树时，读者很容易产生这样一个疑惑：这样复杂的结构和操作，究竟是怎样想到的？事实上，如果把红黑树的红结点并入它的黑父亲，就能看到一個更直观的多路搜索树。

在红黑树中，每个黑结点可以有零个、一个或两个红孩子，而红孩子不能再有红孩子，所以合并后恰好得到含一、二或三个键的结点，分别有二、三或四个孩子位置。图中画出三种典型情况，有一个红右孩子时与中间的图左右对称。

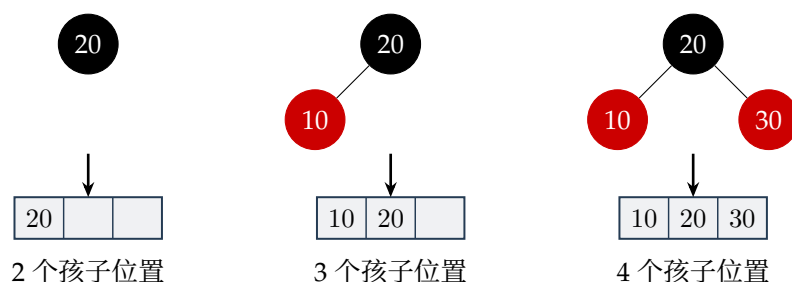


图 2.18 黑色结点与红色孩子合并成一个多键结点。

这样合并之后，原结点的子树也很容易处理：例如右边一组，原来的 10 和 30 的四个子树的元素依次位于 10 左边、10 与 20 之间、20 与 30 之间、30 右边，合并后正好落在多键结点的四个范围里，因此树的搜索次序保留。进一步地，由于红黑树要求从根到 NIL 的路径包含相同个数的黑结点，因此合并后所有叶结点的深度都一致。因此如上的合并操作自然地将红黑树映射到 2-3-4 B 树（因为内部结点也可以存放值，所以是 B 树而不是 B+ 树）。

反过来，一个含三个键 $a < b < c$ 的 2-3-4 B 树结点可以展开为黑 b 带红孩子 a, c ，含两个键的结点可展开为黑父亲带一个红孩子，含一个键的结点则展开为黑结点。逐层展开后，按将红黑树对应到 2-3-4 B 树的过程反向进行，连接逐层的结点，即可得到满足红黑树定义的二叉树。这样就建立了红黑树与 2-3-4 B 树之间的对应关系。当然，同一棵 2-3-4 B 树可能对应不同的红黑树，因为双键结点中的任意一个键都可以作为黑色父结点。如图 2.19 所示。

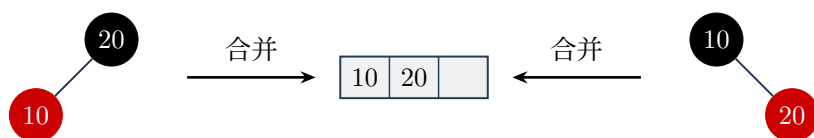


图 2.19 同一个双键 B 树结点的两种红黑树表示。图中省略了黑色 NIL 结点。

再看红黑树和 2-3-4 B 树操作的对应关系。下面三个例子分别说明插入时的染色、单旋和双旋和 B 树操作的对应关系。事实上删除也有类似的对应关系，读者可以自行思考。

例 1：染色 图 2.20 插入 5 后，从 B 树来看，这次插入需要把四个键 5, 10, 20, 30 分开，并将 20 提升为新根。将这一结果对应到红黑树，正好就是插入情况 1 的染色操作。

例 2：单旋 图 2.21 插入 10 后，B 树只需把它加入原结点。但在二叉表示中，根和颜色发生了变化，而这一变化正好对应于红黑树插入的情况 3 的旋转和染色。

例 3：双旋 图 2.22 插入 20 后，B 树仍然只需把它加入原结点。但在二叉表示中，根和颜色发生了变化，而这一变化正好对应于红黑树插入的情况 2 和情况 3 的旋转和染色。

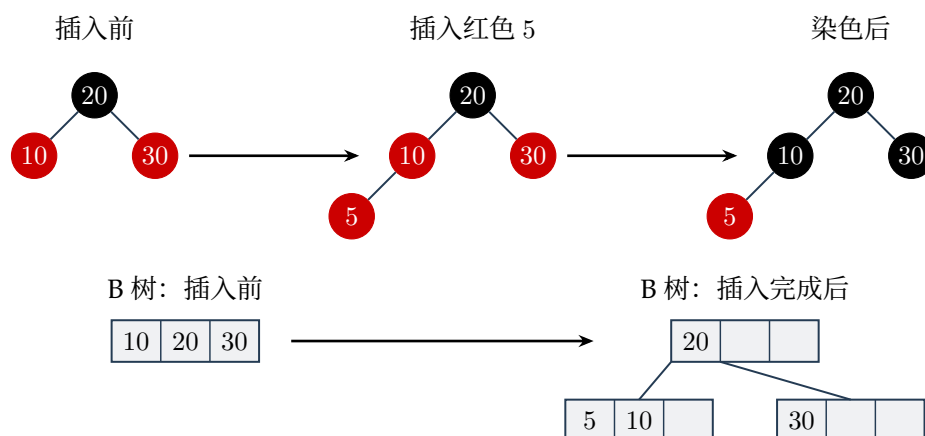


图 2.20 插入 5：染色将原来的满结点分开，键 20 成为新根。

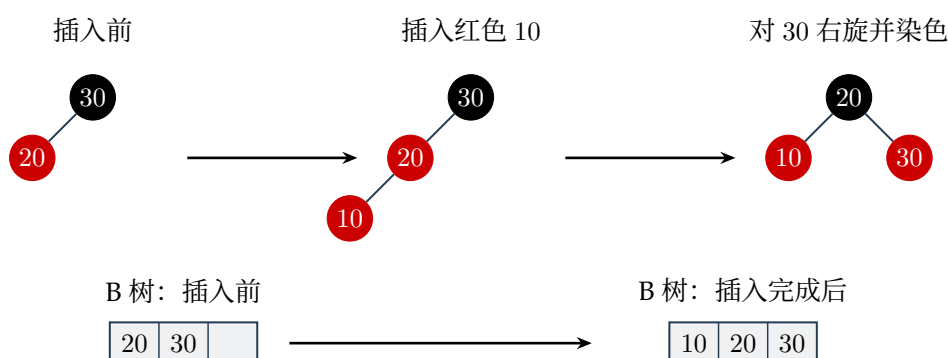


图 2.21 插入 10：一次右旋配合染色，完成三键结点的红黑表示。

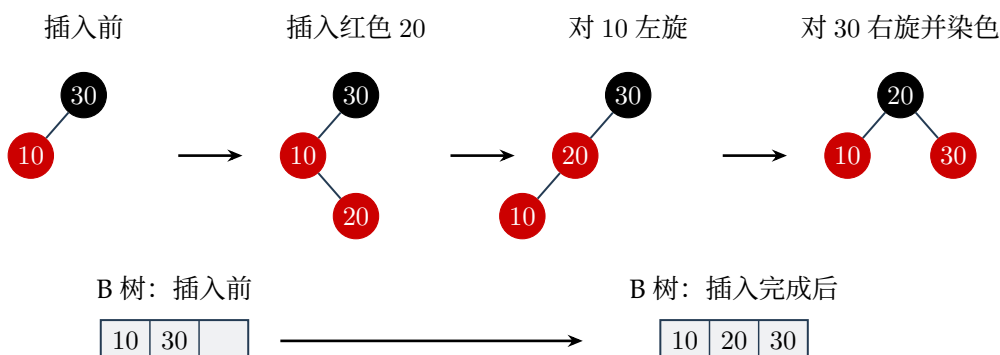


图 2.22 插入 20：先左旋、再右旋并染色，得到与上例相同的三键结点表示。

Guibas 与 Sedgwick 在 1978 年的 *A Dichromatic Framework for Balanced Trees* 中用双色框架研究平衡树，把多路平衡结构与二叉表示联系起来。上面的对应提供了一种理解这一设计的途径。

2.4 习题

习题 2.1 (最长与最短路径). 证明：在一棵红黑树中，从某结点 X 到其后代叶结点 (NIL) 的所有简单路径中，最长的一条路径的长度至多是最短一条的 2 倍。

习题 2.2 (插入的性质). 按照本讲介绍的插入算法，从空树开始连续插入 $n(n > 1)$ 个结点得到一棵红黑树（每一步插入都要保证红黑树性质），试问这棵树一定会有红色结点吗？若是，请给出清晰的证明；若不是，请举出反例。

习题 2.3 (插入后再删除). 考虑将一个结点 X 插入红黑树 T_0 , 得到红黑树 T_1 , 然后紧接着下一步操作又立刻将 X 从 T_1 删除得到 T_2 , 请问 T_0 和 T_2 是否一定一样? 若是, 请给出清晰的证明; 若不是, 请举出反例。

习题 2.4 (合法的红黑树). 设计算法, 检查一棵带颜色的二叉树是否是红黑树。

习题 2.5 (合法的 B+ 树). 设计算法, 验证一棵树是否是 M 阶 B+ 树。

2.5 参考答案

习题 2.1. 这一结论并不难证明: 统计路径上的结点时不计 X , 但计入末端 NIL。每条路径都有 $b = bh(X)$ 个黑结点, 又因为红结点不能相邻, 红结点数至多为 b , 所以路径长度介于 b 和 $2b$ 之间。因此最长路径至多是最短路径的两倍。□

习题 2.2. 答案是一定会, 可以使用数学归纳法证明: $n = 2$ 时显然正确; 此后如果不需要调整, 因为插入的是红色结点, 因此红色结点只可能变多; 如果需要调整, 则根据三种情况的讨论, 无论哪一种情况, 在调整之后一定还保留着红色结点。有读者可能会质疑, 情况 1 如果 G 到了根结点, 则需要被染黑, 但要注意的是, 此时调整前的 X 仍是红色的, 染黑根不会影响它。□

习题 2.3. 不一定, 如下图所示。□

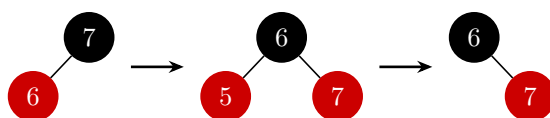


图 2.23 插入 5 后再删除 5, 键集合恢复, 树形改变。

习题 2.4. 略。□

习题 2.5. 略。□

参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第二讲。
2. Cormen 等, 《算法导论》, 第三版, 第 13、18 章。红黑树性质及修复依据第 13 章, B 树对照见第 18 章, 但注意 B 树的定义与本讲 B+ 树略有不同。
3. Guibas、Sedgwick, [A Dichromatic Framework for Balanced Trees](#), FOCS 1978, 8–21。